Technical Specification: GPGPU-Ready Genetic Algorithm

Architecture for the Symmetric Traveling Salesman Problem

Joel Maldonado
Department of Applied Mathematics, University of Arizona
joelmaldonado@arizona.edu

April 2026 | Version 1.0

Abstract

This document specifies the software architecture for a modular, sequential Genetic Algorithm (GA) targeting the Traveling Salesman Problem (TSP), implemented in standard C99. Although the initial execution target is a single CPU core, every architectural decision is made with NVIDIA CUDA in mind: memory layouts prefer coalescing patterns, all functions are stateless with respect to global storage, random-number generators are per-individual, and evaluation is batch-oriented to map directly onto kernel launches. The document covers the formal problem definition, layer decomposition, data-structure design with AoS/SoA analysis, genetic operator specifications, a formal algorithm statement, complexity characterization, and a checklist of CUDA-readiness constraints.

Contents

1 Problem Formalization **1**

2 System Architecture **2**

3 Data Structures and Memory Layout **2**
3.1 Distance Matrix . . . 2
3.2 Tour Representation . . . 2
3.3 Population Layout: AoS vs. SoA . . . 3

4 Genetic Operators **3**
4.1 Initialization . . . 3
4.2 Fitness Evaluation . . . 4
4.3 Selection: Binary Tournament . . . 4
4.4 Crossover: Order Crossover (OX1) . . . 4
4.5 Mutation: Swap and Inversion . . . 4
4.6 Elitism . . . 5

5 Algorithm Statement **5**

6 Complexity Analysis 5

7 CUDA Readiness Constraints 6

8 Validation Layer 7

9 Summary 7

1. Problem Formalization

The Symmetric Traveling Salesman Problem is defined on a complete weighted graph $G = (V, E)$ where $|V| = n$. The distance matrix $D \in \mathbb{R}^{n \times n}$ satisfies $d_{ij} = d_{ji} \geq 0$ and $d_{ii} = 0$. Given the coordinates (x_k, y_k) for city k, the Euclidean distance is

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

A *tour* is a bijection $\pi : \{0, \ldots, n-1\} \to \{0, \ldots, n-1\}$ encoding a Hamiltonian cycle. The objective is

$$\min_{\pi \in S_n} L(\pi) = \sum_{k=0}^{n-2} d_{\pi_k, \pi_{k+1}} + d_{\pi_{n-1}, \pi_0}, \tag{1}$$

where S_n denotes the symmetric group on n elements. Because L is symmetric in d, the search space has effective size $(n-1)!/2$.

2. System Architecture

The implementation is decomposed into five loosely-coupled layers. Each layer exposes a well-defined functional interface and holds no mutable global state, a constraint that directly enables CUDA kernel extraction.

3. Data Structures and Memory Layout

3.1. Distance Matrix

The matrix D is allocated once at startup and treated as read-only for the entirety of GA execution. It is stored in row-major order as a flat array of `double`, indexed by $i \cdot n + j$.

HPC Note: For small instances ($n \leq 256$, i.e., $\leq 512\,\text{KiB}$ at 64-bit), D fits in CUDA constant memory and benefits from broadcast reads when all threads in a warp address the same entry. For larger instances it must reside in global memory; a symmetric half-matrix layout ($n(n+1)/2$ elements) reduces footprint and may improve cache utilization.

Table 1: Layer definitions and forward-looking GPU porting strategies.

Layer	Responsibility	CUDA Porting Strategy
Instance	Coordinate I/O; distance matrix construction	Pin D to constant memory ($n \leq 256$) or L2-cached global memory
Representation	Population and tour memory management	Migrate AoS layout to SoA for coalesced access
Evaluation	Fitness computation; tour validation	One thread per individual; parallel reduction within tour
Evolution	Selection, crossover, mutation, elitism	Warp-level primitives (`__shfl_sync`) for tournament logic
Control	GA main loop; termination criteria	Remains host-side; issues asynchronous kernel launches

3.2. Tour Representation

A tour is a permutation stored as an integer array of length n. The associated struct carries precomputed fitness to avoid redundant evaluation:

```
typedef struct {
    int    *cities;    /* permutation of {0, ..., n-1}          */
    double  length;    /* total Euclidean tour length            */
    double  fitness;   /* derived from length; higher is better */
} Tour;
```

Listing 1: Sequential tour struct (AoS element).

Invariant. After any genetic operator, every valid tour π must satisfy $\{\pi_0, \ldots, \pi_{n-1}\} = \{0, \ldots, n-1\}$. Violations indicate a logic error and are caught by the validation layer (Section 8).

3.3. Population Layout: AoS vs. SoA

The sequential implementation uses an *Array of Structures* (AoS), which is natural for single-threaded access patterns.

```
Tour population[POP_SIZE];
/* Thread i accesses population[i].cities[k]:
   addresses stride by sizeof(Tour) -- non-coalesced. */
```

Listing 2: AoS: simple but GPU-hostile.

GPU Constraint: On a GPU, when thread i and thread $i+1$ access `population[i].cities[k]` and `population[i+1].cities[k]` respectively, the hardware must issue W independent 32-byte transactions for a warp of $W = 32$ threads because the `cities` pointers are separated by `sizeof(Tour)` bytes. This fully serializes what should be a single coalesced load.

The GPU-ready layout separates gene arrays from metadata:

```
/* All gene-k values for the entire population are contiguous. */
int    *pop_cities;    /* [POP_SIZE * n]  row i: cities of individual i */
double *pop_length;    /* [POP_SIZE] */
double *pop_fitness;   /* [POP_SIZE] */

/* Thread i reads gene k: pop_cities[i*n + k].
   Threads 0..31 reading k=0 hit addresses {0,n,2n,...,31n} --
   still strided.  Transpose to [n * POP_SIZE] with layout
   pop_cities[k*POP_SIZE + i] for fully coalesced access. */
```

Listing 3: SoA: GPU-friendly, migration target.

The transposed SoA layout (`pop_cities[k * POP_SIZE + i]`) ensures that a warp reading gene k of all individuals issues a single coalesced 128-byte transaction, reducing memory traffic by up to 32×.

4. Genetic Operators

4.1. Initialization

The initial population is generated by Fisher–Yates shuffles seeded from a per-individual RNG state. Diversity is guaranteed by construction; no duplicate-rejection loop is needed.

4.2. Fitness Evaluation

Tour length is computed directly from Equation (1) using the precomputed matrix D. Fitness is defined as the inverse of length, $f(\pi) = 1/L(\pi)$, so that higher fitness corresponds to shorter tours.

Parallelization structure. Evaluation is *embarrassingly parallel* across the population: individual i's fitness depends only on its own permutation and the read-only matrix D. A future CUDA kernel assigns one thread block per individual, with each thread summing a contiguous segment of edges and a final warp reduction collecting the total.

4.3. Selection: Binary Tournament

For each offspring slot, two individuals are drawn uniformly at random and the fitter one is selected. Tournament size $k = 2$ is the default; k can be tuned to control selection pressure without restructuring the algorithm.

HPC Note: Tournament selection requires no global information: each contest is $O(k)$ work and completely independent. This contrasts with fitness-proportionate (roulette-wheel) selection, which requires an $O(N \log N)$ prefix scan over the entire population — a bottleneck even with GPU-accelerated prefix sums.

4.4. Crossover: Order Crossover (OX1)

Order Crossover preserves the relative ordering of cities inherited from one parent while filling remaining positions from the other. The algorithm proceeds in three steps: (1) copy a contiguous segment $[\ell, r]$ from parent A directly into the offspring; (2) traverse parent B starting at position $r + 1 \pmod{n}$ and append cities not already present; (3) wrap to fill all positions.

The sequential implementation uses an $O(n)$ boolean lookup buffer to test membership. The planned GPU implementation replaces this with a 64-bit bitmask held in registers (valid for $n \leq 64$) or shared memory (valid for $n \leq 1024$), eliminating branch divergence caused by cache-miss-dependent conditional branches in the linear scan.

GPU Constraint: OX1 contains data-dependent branching proportional to n, which can cause warp divergence when different threads reach different branch outcomes at the same instruction. Mitigation: predicated instructions and bitmask membership tests that resolve without branching.

4.5. Mutation: Swap and Inversion

Two mutation operators are supported:

- **Swap**: select two positions $i \neq j$ uniformly at random; exchange $\pi_i \leftrightarrow \pi_j$. Complexity: $O(1)$.
- **Inversion (2-opt move)**: select a sub-segment $[\ell, r]$; reverse it in place. Complexity: $O(n)$.

Each operator is applied independently per individual; there is no inter-individual communication, making mutation trivially parallel.

GPU Constraint: Mixing operator types within a single kernel launch introduces warp divergence if different threads branch to different operators. Prefer separate kernel launches per operator type, or compile a single-operator kernel and execute them sequentially on the host.

4.6. Elitism

The top e individuals (by fitness) are copied unchanged into the next generation before new offspring are generated. This prevents fitness regression across generations.

For the sequential baseline, elitism is implemented via a partial sort (introselect) in $O(N)$ expected time. On the GPU, a parallel Top-e selection via bitonic sort on the fitness array is preferred over a full sort to minimize kernel occupancy.

5. Algorithm Statement

Algorithm 1: Sequential Reference GA for TSP

Input: TSP instance I (coordinates, n); GA configuration C (population size N, elite count e, tournament size k, crossover rate p_c, mutation rate p_m, maximum generations G)

Output: Best tour π^* found

```
D ← BuildDistanceMatrix(I)
P ← InitializePopulation(N, n)
EvaluateFitness(P, D)
for g ← 1 to G do
    P′ ← ExtractElites(P, e)
    while |P′| < N do
        a ← Tournament(P, k)
        b ← Tournament(P, k)
        if Uniform(0, 1) < p_c then
            c ← OX1Crossover(a, b)
        else
            c ← Copy(a)
        end
        if Uniform(0, 1) < p_m then
            Mutate(c)
        end
        Append(P′, c)
    end
    P ← P′
    EvaluateFitness(P, D)
    LogStatistics(g, P)
end
return arg max_{π∈P} f(π)
```

6. Complexity Analysis

Let N denote population size, n the number of cities, G the number of generations, e the elite count, and k the tournament size.

The total wall-clock complexity for the sequential implementation is $O(GNn)$. A GPU implementation with N threads in parallel reduces the dominant evaluation and crossover terms from $O(Nn)$ to $O(n)$ per generation, yielding a theoretical N-fold speedup on those kernels.

Table 2: Per-generation computational complexity.

Operation	**Sequential**	**Parallelizable?**
Distance matrix build	$O(n^2)$ once	Yes, $O(n)$ with n threads
Fitness evaluation	$O(Nn)$	Yes, $O(n)$ with N threads
Tournament selection	$O(kN)$	Yes, $O(k)$ with N threads
OX1 crossover	$O(Nn)$	Yes, $O(n)$ with N threads
Swap mutation	$O(N)$	Yes, $O(1)$ with N threads
Inversion mutation	$O(Nn)$	Yes, $O(n)$ with N threads
Elite extraction	$O(N)$	Partial (bitonic sort)
Total per generation	$O(GNn)$	Reduces to $O(Gn)$ on GPU

7. CUDA Readiness Constraints

The following constraints are enforced in the sequential baseline to ensure that GPU porting is an incremental refactor rather than a rewrite.

CR-1. Zero global mutable state. No `static`, `extern`, or file-scope mutable variables are used. All data flows through explicit pointer arguments. This is a hard requirement because CUDA device functions have no access to host address space.

CR-2. Per-individual RNG state. Each individual carries its own PRNG state (a `uint64_t` for a splitmix64 generator). On the GPU, this maps directly to a `curandState` per thread, avoiding the serialization caused by a single shared generator.

CR-3. Batch-oriented evaluation interface. The evaluation function signature is `evaluate_population(Tour* int N, double* D, int n)`, not a loop over `evaluate_tour` calls. The batch signature mirrors a CUDA kernel launch: the host passes a device pointer and the GPU fills results in parallel.

CR-4. Flat, pointer-based memory. No nested dynamic allocation (`Tour**` with per-element `malloc`). All population data is held in contiguous flat buffers compatible with `cudaMalloc` and `cudaMemcpy`.

CR-5. Deterministic seeding. A command-line seed fully determines all random choices, enabling exact reproduction of any run. This is critical for isolating GPU race conditions: compare GPU output against the seeded sequential baseline.

CR-6. SoA migration path documented. The AoS-to-SoA transformation described in Section 3.3 is the first step of the CUDA port. No other algorithmic change is required before writing kernel code.

8. Validation Layer

Correctness of permutation operators is verified by a dedicated validator executed after every crossover and mutation in debug builds (`#ifdef DEBUG`).

Permutation check. An $O(n)$ pass marks each city encountered in the tour; any duplicate or missing index raises an assertion:

```
bool validate_tour(const Tour *t, int n) {
    bool seen[n];
    memset(seen, 0, n * sizeof(bool));
    for (int k = 0; k < n; k++) {
        if (t->cities[k] < 0 || t->cities[k] >= n) return false;
        if (seen[t->cities[k]])                    return false;
        seen[t->cities[k]] = true;
    }
    return true;
}
```

Listing 4: Tour validation.

Fitness consistency check. After evaluation, $L(\pi)$ is independently recomputed and compared against the stored value within a relative tolerance of 10^{-9}.

Relevance to GPU debugging. The validator provides a ground-truth reference against which GPU kernel output can be checked element-wise. Memory corruption from out-of-bounds writes in a CUDA kernel typically manifests as a permutation invariant violation before causing a numerical anomaly, making the validator an early-warning system for kernel bugs.

9. Summary

This specification defines a sequential GA for the TSP whose architecture is shaped by the eventual requirements of a CUDA implementation. The key design decisions are:

- **Stateless functions** to enable direct kernel extraction.
- **Flat contiguous memory** compatible with device allocation.
- **Batch evaluation interface** mirroring kernel launch semantics.
- **Per-individual RNG** ready for `cuRAND` substitution.
- **Documented AoS-to-SoA transition** as the primary migration step.
- **Tournament selection** to avoid global prefix-sum dependencies.

The result is a CPU baseline that can be validated rigorously and then ported to GPU by replacing the evaluation and evolution loops with kernel launches — no structural redesign required.